
DBMS COURSE PROJECT

Scholarship & Financial Aid Management System

Project Report

Name	Shailesh Charan
Roll Number	25WU0102252
Course	B.Tech CSE (AI & ML)
University	Woxsen University, Hyderabad
Faculty Name	Dr. Kiran Mayee Adavala
Academic Year	2026–2027

2. Abstract

The Scholarship & Financial Aid Management System is a database-driven desktop application designed to manage student information, scholarship records and scholarship applications in a structured manner. The system uses Python with Tkinter for the graphical user interface and MySQL for persistent data storage. The database contains related entities such as Department, Course, Student, Donor, Scholarship, Eligibility_Criteria, Application, Document, Staff_Admin, Disbursement, Payment and Donation. Primary keys and foreign keys are used to maintain referential integrity. The application provides viewing, insertion and updating operations through a simple desktop UI, while SQL queries support reporting and analysis.

3. Introduction and Problem Statement

Scholarship administration involves maintaining student details, scholarship eligibility, donor information, applications and financial transactions. If these records are handled manually or in disconnected files, duplicate data, inconsistent updates and difficulty in tracking applications can occur.

The proposed system centralizes these records in a relational database. A Python/Tkinter application provides a user-friendly interface to view and manage student and scholarship application records, while MySQL enforces relationships between entities through primary-key and foreign-key constraints.

Problem Statement: Design and implement a relational database management system for scholarship and financial-aid administration that stores related records, maintains data integrity, supports useful SQL queries, and provides a working user interface connected to MySQL.

4. Objectives and Scope

Objectives

- Design a relational database for scholarship and financial-aid management.
- Store student, course, department, donor, scholarship and application information.
- Maintain referential integrity using primary and foreign keys.
- Provide a Tkinter-based desktop UI connected to MySQL.
- Support viewing, adding and updating student/application records.
- Execute SQL queries involving filtering, joins, aggregation and grouping.
- Provide a maintainable structure that can be extended with additional financial workflows.

Scope

The current project covers database creation, sample data population, relational querying and a desktop UI for student and scholarship application management. The database design also includes document, staff, disbursement, payment and donation entities for a broader financial-aid workflow.

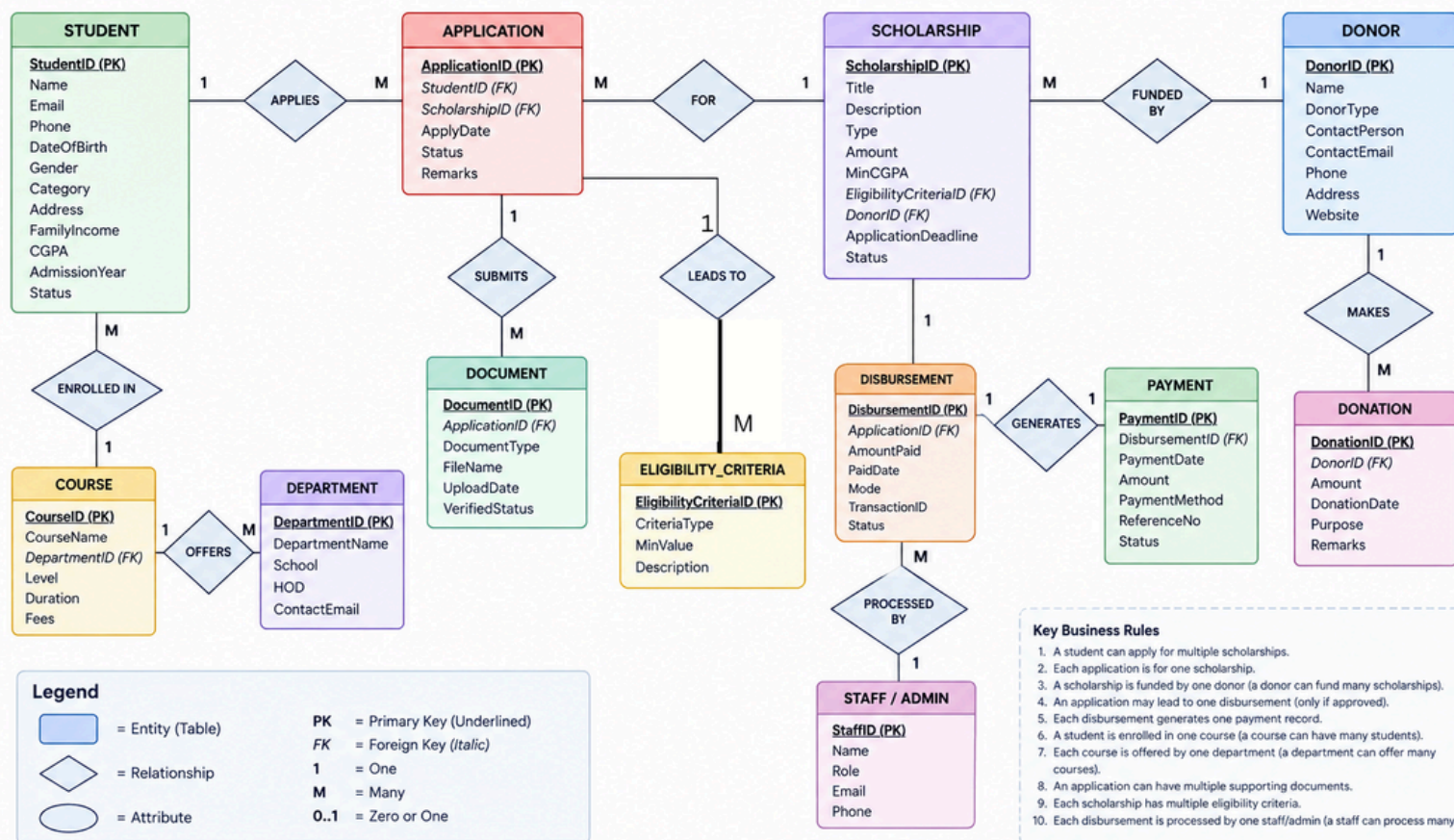
5. Software and Hardware Requirements

Category	Requirement
Operating System	Windows (development environment)
Programming Language	Python 3.x
GUI Framework	Tkinter / ttk
Database	MySQL
Connector	mysql-connector-python
IDE	Visual Studio Code
Version Control	Git and GitHub
Processor	Modern x64 processor
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free for project/tools

6. ER Diagram

The ER model represents the main entities and their relationships. Department is related to Course; Course is related to Student; Student and Scholarship are connected through Application; Scholarship is related to Donor and Eligibility_Criteria; Application is related to Document and Disbursement; Disbursement is related to Payment and Staff_Admin; Donor is related to Donation.

Scholarship & Financial Aid Management System Entity – Relationship Diagram (Enhanced)



A complete ER model for managing scholarships — from student eligibility and application to donor funding and disbursement.

7. Relational Schema and Normalization

Relation	Primary Key	
Department	DepartmentID	—
Course	CourseID	DepartmentID → Department
Student	StudentID	CourseID → Course
Donor	DonorID	—
Scholarship	ScholarshipID	DonorID → Donor; EligibilityCriteriaID → Eligibility_Criteria
Eligibility_Criteria	EligibilityCriteriaID	ScholarshipID → Scholarship
Application	ApplicationID	StudentID → Student; ScholarshipID → Scholarship
Document	DocumentID	ApplicationID → Application
Staff_Admin	StaffID	—
Disbursement	DisbursementID	ApplicationID → Application; StaffID → Staff_Admin
Payment	PaymentID	DisbursementID → Disbursement
Donation	DonationID	DonorID → Donor

Normalization

The design separates independent concepts into their own relations and links them using keys. Attributes are atomic within their respective tables, satisfying the intent of 1NF. Non-key attributes are associated with the key of their own relation, supporting 2NF. Transitive dependencies are reduced by separating entities such as Department, Course, Donor, Scholarship, Application and financial transaction tables, supporting a 3NF-oriented design.

8. Data Dictionary

Table	Column	Data Type	Constraint / Role
Department	DepartmentID	INT	PK, AUTO_INCREMENT
Department	DepartmentName	VARCHAR(100)	NOT NULL
Department	School	VARCHAR(100)	Optional
Course	CourseID	INT	PK, AUTO_INCREMENT
Course	DepartmentID	INT	FK, NOT NULL
Course	CourseName	VARCHAR(100)	NOT NULL
Student	StudentID	INT	PK, AUTO_INCREMENT
Student	Name	VARCHAR(100)	NOT NULL
Student	Email	VARCHAR(100)	UNIQUE
Student	Phone	VARCHAR(20)	Optional
Student	DateOfBirth	DATE	Optional
Student	Gender	VARCHAR(20)	Optional
Student	Category	VARCHAR(50)	Optional
Student	Address	VARCHAR(255)	Optional
Student	FamilyIncome	DECIMAL(12,2)	Optional
Student	CGPA	DECIMAL(4,2)	Optional
Student	AdmissionYear	YEAR	Optional
Student	Status	VARCHAR(50)	Optional
Student	CourseID	INT	FK → Course
Donor	DonorID	INT	PK, AUTO_INCREMENT
Donor	Name	VARCHAR(100)	NOT NULL
Donor	DonorType	VARCHAR(50)	Optional
Scholarship	ScholarshipID	INT	PK, AUTO_INCREMENT
Scholarship	Title	VARCHAR(150)	NOT NULL
Scholarship	Description	TEXT	Optional
Scholarship	Type	VARCHAR(50)	Optional
Scholarship	Amount	DECIMAL(12,2)	Optional
Scholarship	MinCGPA	DECIMAL(4,2)	Optional
Scholarship	DonorID	INT	FK → Donor
Scholarship	ApplicationDeadline	DATE	Optional
Scholarship	Status	VARCHAR(50)	Optional
Application	ApplicationID	INT	PK, AUTO_INCREMENT
Application	StudentID	INT	FK → Student, NOT NULL
Application	ScholarshipID	INT	FK → Scholarship, NOT NULL
Application	ApplyDate	DATE	Optional
Application	Status	VARCHAR(50)	Optional
Application	Remarks	TEXT	Optional

9. SQL Commands Used (DDL and DML) with Sample Outputs

DDL examples

```
CREATEDATABASEScholarshipFinancialAidDB;
USE ScholarshipFinancialAidDB;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(20),
    DateOfBirth DATE,
    Gender VARCHAR(20),
    Category VARCHAR(50),
    Address VARCHAR(255),
    FamilyIncome DECIMAL(12,2),
    CGPA DECIMAL(4,2),
    AdmissionYear YEAR,
    Status VARCHAR(50),
    CourseID INT,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

DML examples

```
INSERT INTO Student
(Name, Email, Phone, DateOfBirth, Gender, Category,
Address, FamilyIncome, CGPA, AdmissionYear, Status, CourseID)
VALUES
('Aarav Sharma', 'aarav@gmail.com', '9876500001',
'2007-04-12', 'Male', 'General',
'Hyderabad', 450000, 8.70, 2026, 'Active', 1);

UPDATE Student
SET CGPA = 8.80
WHERE StudentID = 1;

SELECT * FROM Student;
```

Sample output: The database returns the inserted student row when `SELECT * FROM Student;` is executed. The project SQL script also includes INSERT statements for departments, courses, donors, scholarships, eligibility criteria, applications, documents, staff, disbursements, payments and donations.

10. Queries with Outputs

Query 1 – Students from Bangalore with donor, scholarship title and department

```
SELECT
    Student.Name          AS      Student_Name,
    Donor.Name            AS      Donor_Name,
    Scholarship.Title      AS      Scholarship_Title,
    Department.DepartmentName AS Department
FROM Student
JOIN Course ON Student.CourseID = Course.CourseID
JOIN Department ON Course.DepartmentID = Department.DepartmentID
JOIN Application ON Student.StudentID = Application.StudentID
JOIN Scholarship ON Application.ScholarshipID = Scholarship.ScholarshipID
JOIN Donor ON Scholarship.DonorID = Donor.DonorID
WHERE Student.Address = 'Bangalore';
```

Student_Name	Donor_Name	Scholarship_Title	Department
Megha Nair	Wipro Foundation	SC Education Scholarship	Biotechnology

This is the Presentation-II query included in the project SQL work. The sample data contains one Bangalore student, Megha Nair, and the corresponding joins produce the row shown above.

Query 2 – Approved applications

```
SELECT
    Student.Name AS Student_Name,
    Scholarship.Title,
    Application.Status
FROM Application
JOIN Student ON Application.StudentID = Student.StudentID
JOIN Scholarship ON Application.ScholarshipID = Scholarship.ScholarshipID
WHERE Application.Status = 'Approved';
```

Student_Name	Scholarship	Status
Aarav Sharma	Academic Excellence Scholarship	Approved
Priya Reddy	Women in Technology Scholarship	Approved
Sneha Rao	Academic Excellence Scholarship	Approved
Vikram Singh	ST Student Support Scholarship	Approved
Karthik Reddy	Innovation Scholarship	Approved

Query 3 – Students with CGPA above 8.5

```
SELECT Name, CGPA, Category, Address
FROM Student
WHERE CGPA > 8.5;
```

Name	CGPA	Category	Address
Aarav Sharma	8.70	General	Hyderabad
Priya Reddy	9.10	OBC	Warangal
Sneha Rao	9.30	General	Hyderabad
Ananya Patel	8.80	OBC	Secunderabad
Megha Nair	9.00	SC	Bangalore

11. UI Design and Screenshots

The user interface is implemented using Tkinter. The main window provides navigation to view students, add and update students, view scholarship applications, add applications, delete applications and update applications.

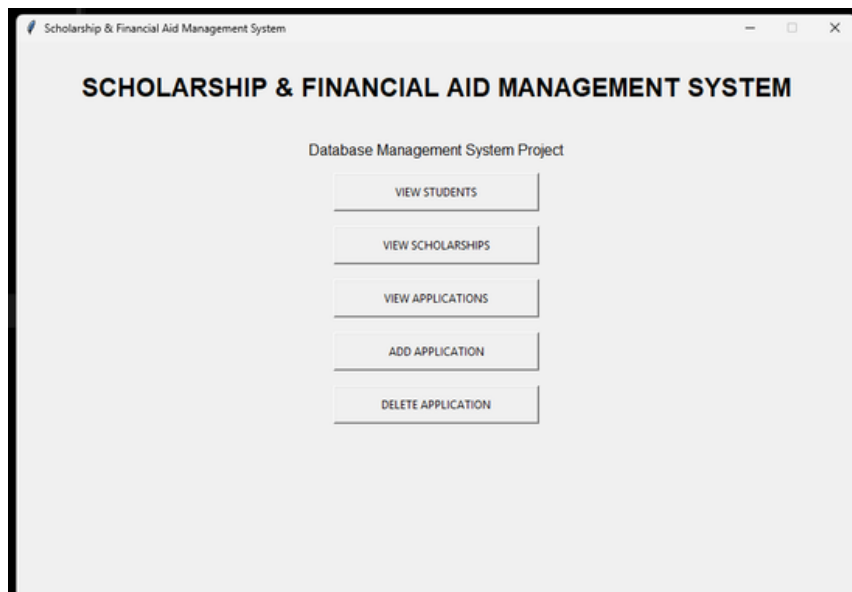


Figure 2. Main application window.

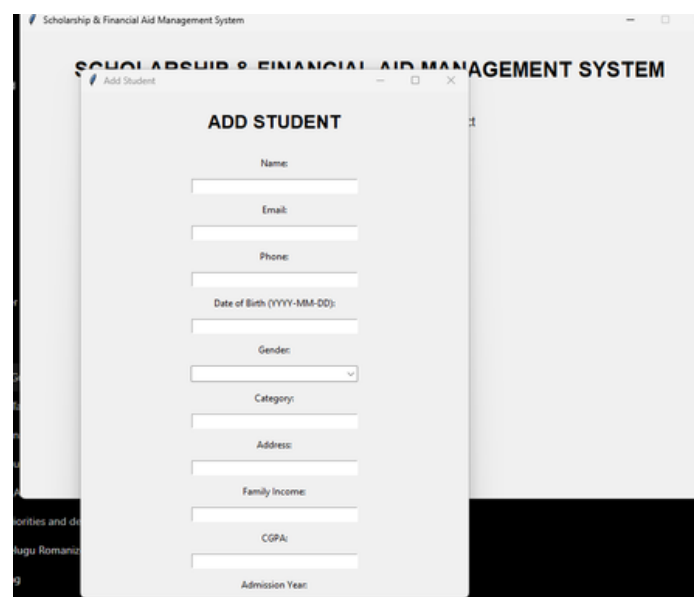
The screenshot shows a smaller window titled "Add Student" overlaid on the main application window. The "Add Student" window has a title bar and a close button. Inside, the title "ADD STUDENT" is centered. Below it, there are nine input fields: "Name:", "Email:", "Phone:", "Date of Birth (YYYY-MM-DD):", "Gender:" (with a dropdown arrow), "Category:", "Address:", "Family Income:", "CGPA:", and "Admission Year:". The main application window is visible in the background, slightly faded.

Figure 3. Add Student form.

ADD APPLICATION

Student ID:

Scholarship ID:

Apply Date (YYYY-MM-DD):

Status:

Remarks:

SAVE APPLICATION

Figure 4. Add Application form.

STUDENT RECORDS

StudentID	Name	Email	Phone	DateOfBirth	Gender	Category	Address	FamilyIncome	CGPA	AdmissionYear	Status
1	Aarav Sharma	aarav@gmail.com	9876500001	2007-04-12	Male	General	Hyderabad	450000.00	8.70	2026	Active
2	Priya Reddy	priya@gmail.com	9876500002	2007-08-21	Female	OBC	Warangal	350000.00	9.10	2026	Active
3	Rahul Kumar	rahul@gmail.com	9876500003	2006-12-15	Male	SC	Vijayawada	280000.00	8.20	2026	Active
4	Sneha Rao	sneha@gmail.com	9876500004	2007-02-10	Female	General	Hyderabad	600000.00	9.30	2026	Active
5	Vikram Singh	vikram@gmail.com	9876500005	2006-11-05	Male	ST	Nizamabad	220000.00	7.90	2026	Active
6	Ananya Patel	ananya@gmail.com	9876500006	2007-06-18	Female	OBC	Secunderabad	320000.00	8.80	2026	Active
7	Karthik Reddy	karthik@gmail.com	9876500007	2007-09-25	Male	General	Karimnagar	500000.00	8.50	2026	Active
8	Megha Nair	megha@gmail.com	9876500008	2006-10-30	Female	SC	Bangalore	250000.00	9.00	2026	Active

Figure 5. Student records view.

APPLICATION RECORDS

ApplicationID	StudentID	ScholarshipID	ApplyDate	Status	Remarks
1	1	1	2026-09-01	Approved	Excellent academic performance
2	2	2	2026-09-02	Approved	Eligible under women in technology category
3	3	5	2026-09-03	Under Review	Documents submitted
4	4	1	2026-09-04	Approved	High CGPA
5	5	4	2026-09-05	Approved	ST category verified
6	6	6	2026-09-06	Under Review	Income documents submitted
7	7	8	2026-09-07	Approved	Innovation project submitted
8	8	5	2026-09-08	Pending	Verification in progress
9	1	1	2026-10-05	Pending	Test application

Figure 6. Application records view.

12. Implementation Details

Technology Stack

- Python – application logic.
- Tkinter / ttk – desktop graphical user interface.
- MySQL – relational database management system.
- mysql-connector-python – Python-to-MySQL connectivity.
- Visual Studio Code – development environment.
- Git/GitHub – version control and project submission.

Database Connectivity

```
importmysql.connector

def connect_database():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="<configured locally>",
        database="ScholarshipFinancialAidDB"
    )
```

Key UI/database operation

```
cursor.execute(
    "SELECT * FROM Student"
)
students = cursor.fetchall()
```

The application opens a database connection, creates a cursor, executes SQL statements and commits changes after INSERT/UPDATE operations. Error handling uses `mysql.connector.Error` and displays database errors through Tkinter message boxes.

13. Testing – Test Cases and Results

Test ID	Test Action	ExpectedResult	Result
TC01	Launch application	Main window opens	Pass
TC02	View students	Student records displayed from MySQL	Pass
TC03	Add student with valid data	New student inserted	Pass
TC04	Update existing student	Selected student details updated	Pass
TC05	View applications	Application records displayed	Pass
TC06	Add application with valid StudentID/ScholarshipAlpDplication inserted		Pass
TC07	Add application with non-existent StudentID	Foreign-key error displayed	Pass – integrity constraint worked
TC08	Run SQL join query	Joined result returned	Pass
TC09	Python/MySQL connectivity test	Connection test returned True	Pass
TC10	Run application after dependency installation	Tkinter app launches successfully	Pass

During development, an application insert using an invalid StudentID produced a MySQL foreign-key error. This was expected database behavior and confirmed that referential integrity between Application.StudentID and Student.StudentID is enforced.

14. Conclusion and Future Enhancements

The Scholarship&Financial AidManagement System demonstrates the design and implementation of a relational database application using MySQL and Python/Tkinter. The project separates core entities into related tables, uses keys to maintain data integrity, populates the database with sample records, supports SQL-based reporting and provides a working graphical interface for common student and application operations.

Future Enhancements

- Add login and role-based access control for staff and administrators.
- Replace manual StudentID/ScholarshipID entry with dropdowns populated from MySQL.
- Add full CRUD screens for donors, scholarships, documents and financial transactions.
- Add search, filtering and export-to-CSV/PDF features.
- Add automated scholarship eligibility checking based on CGPA, category and income.
- Add dashboard charts for applications, disbursements and donations.
- Move database credentials to environment variables or a secure configuration file.

15. References

- DBMS Course Project–Final Presentation Instructions, Woxsen University, 2026.
- DBMS PPT 2 – Scholarship & Financial Aid Management System: relational design, sample data and query execution.
- Project master MySQL script: ScholarshipFinancialAidDB database creation, table definitions, sample data and SQL queries.
- Project Python source code: Tkinter user interface and mysql-connector-python database connectivity.
- MySQL documentation – SQL data definition, data manipulation and referential integrity concepts.
- Python documentation – Tkinter and mysql-connector-python usage.

16. Appendix – GitHub Repository Link

The project source code, README, requirements file and supporting project material are maintained in the GitHub repository below:

GitHub: <https://github.com/Shaileshcharan2029/ScholarshipSystem>

The current repository contains app.py, app_backup.py, README.md, requirements.txt and .gitignore. For the final course submission, the repository should also contain the required Presentation-I, Presentation-II, Presentation-III and Project-Report folders if required by the faculty instructions.

Submission checklist

- Project report exported as PDF.
- ER diagram included.
- Relational schema and normalization explained.
- Data dictionary included.
- DDL/DML examples and query outputs included.
- UI screenshots included.
- Testing results included.
- GitHub link included.
- Faculty name should be filled on the cover page before submission.